

Evidence-Traceable LLM Reporting for Industrial Process Fault Detection and Diagnosis

Jiyu Chen^a, Wei Peng^{a,*}, Lixin Zhang^b, Mingfu Zhu^c, Yifan Hu^d, Ming Zhai^{a,*}, Rui Xi^e, Tien-Chien Jen^f, Ronghan Wei^{a,b,f,g,*}

^a*School of Mechanics and Safety Engineering, Zhengzhou University, Zhengzhou, 450001, China*

^b*School of Energy and Materials, Shihezi University; Institute of Bingtuan Energy Development Research, Shihezi University; Shihezi University, Shihezi, Xinjiang, 832000, China*

^c*Henan Polytechnic University, Jiaozuo, 454000, Henan, China*

^d*Zhengzhou V3 Biotechnology Co., Ltd., Zhengzhou, 450047, China*

^e*School of Mechanical Engineering, North China University of Water Resources and Electric Power, Zhengzhou, 450001, China*

^f*Department of Mechanical Engineering Science, University of Johannesburg, Cnr Kingsway and University Road, Auckland Park, Johannesburg, 2092, South Africa*

^g*State Key Laboratory of Critical Metal Benefication, Metallurgy and Purification, Zhengzhou University, Zhengzhou, 450001, Henan, P. R. China*

Abstract

Large language models (LLMs) can turn fault-diagnosis outputs into readable reports, but they may also produce evidence-like fields that are not machine-checkable against the current process data. This study examines evidence-traceable LLM reporting for industrial fault detection and diagnosis. The workflow uses an Evidence Citation Schema: diagnostic tools populate an evidence record, fixed report fields are checked against that record, and LLM-generated text is evaluated separately from structured evidence transcription. On the Tennessee Eastman Process benchmark, evidence fidelity is measured with Evidence Field Traceability (EFT), Untraceable Report Rate (URR), and a preliminary rule-based Report Actionability Score (RAS). A passive LLM given all tool outputs in a loose prompt has $URR = 77.1\%$,

*Corresponding authors: Wei Peng (pw2022@zzu.edu.cn), Ming Zhai (mzhai@zzu.edu.cn), and Ronghan Wei (prof.wei@outlook.com).

driven by paraphrased SHAP variable identifiers rather than numeric-field errors, whereas exact-table prompting and validator-regeneration eliminate structured-field violations when the evidence bundle is preassembled. A ReAct implementation, EviFDD-Agent, achieves EFT = 0.997 and URR = 1.4% with DeepSeek-V4-Flash ($n = 210$), but the same protocol is less stable and much slower with DeepSeek-V4-Pro (URR = 19.5%, $11.8\times$ longer wall time). The LLM is therefore positioned above deterministic evidence serialization: critical evidence fields are produced by tools or schema-controlled serializers, while the model is reserved for constrained narrative explanation, recommendation drafting, and auditable tool interaction.

Keywords: fault detection and diagnosis, large language model agent, evidence grounding, Tennessee Eastman Process, ReAct agent, explainable AI

1. Introduction

Industrial process fault detection and diagnosis (FDD) is tied directly to plant safety, equipment protection, and production losses [1, 2, 3]. Classical model-based and data-driven FDD methods—including principal component analysis (PCA), support vector machines, and neural networks—perform well on benchmark datasets such as the Tennessee Eastman Process (TEP) [4, 5]. Their outputs, however, are usually fault labels, statistics, and scores rather than operator-facing explanations.

LLMs offer a natural way to turn such outputs into fault summaries, recommendations, and causal narratives [16, 17]. Early LLM-assisted FDD work has already shown this direction for post-hoc explanation of classifier outputs [19]. The risk is that an LLM report may contain values that look like evidence but are not tied to the current measurement vector or any diagnostic computation. In a small tool-free sanity check, an unconstrained LLM prompted with raw sensor readings has an *Untraceable Report Rate* (URR) of 100%: every report contains at least one required structured field that cannot be traced to tool evidence. This check is used only to illustrate unsupported Type III fields; the matched comparisons use the 210-sample

evidence-based configurations.

The ReAct framework [20] provides a general approach for interleaving LLM generation with tool calls. Tool-augmented LLM systems have been used for tasks including web search, code execution, and database querying [22, 23], but their use in industrial process monitoring requires explicit attention to evidence provenance. ReAct is not treated here as the only or most token-efficient way to enforce schema compliance; its role is an auditable orchestration mechanism for settings where evidence is gathered through sequential diagnostic tools rather than preassembled into a clean table.

EviFDD-Agent is designed around evidence-constrained, tool-augmented reporting. Its main device is an *Evidence Citation Schema*: diagnostic tools populate a structured evidence record, final reports are scored field by field with *Evidence Field Traceability* (EFT), and the LLM is evaluated as an explanation and orchestration layer rather than as a trusted transcriber of critical evidence fields. If the agent tries to finish early, the executor redirects it to the next missing tool.

The contributions are:

1. an Evidence Citation Schema for linking structured diagnostic fields to tool outputs;
2. EFT, URR, and a preliminary rule-based RAS metric for measuring field traceability, report-level traceability failure, and surface-level report completeness;
3. a deployment-oriented boundary between deterministic evidence serialization and constrained LLM narrative generation; and
4. a TEP evaluation of passive, deterministic, schema-controlled, and agent configurations, plus a small tool-free sanity check.

2. Related Work

2.1. Data-Driven Fault Detection and Diagnosis

Fault detection and diagnosis has a long history in chemical and process systems engineering, spanning analytical models, knowledge-based systems,

and data-driven monitoring [1, 2, 3, 6]. With the growth of plant historians and high-dimensional sensor measurements, multivariate statistical process control became a common framework for monitoring continuous and batch processes [7, 8]. Statistical process monitoring using PCA is a foundational approach to FDD [5]. PCA projects high-dimensional sensor data onto a lower-dimensional subspace and detects anomalies via the Hotelling T^2 statistic and the Squared Prediction Error (SPE, or Q -statistic), each tested against control limits derived from the F -distribution or the Jackson–Mudholkar approximation [9]. The Tennessee Eastman Process remains a widely used benchmark for evaluating such methods because it provides a plant-wide nonlinear process simulation with defined fault scenarios [4, 29].

Many recent FDD pipelines combine statistical monitoring with machine-learning classifiers. Isolation Forest [10] gives a density-free anomaly score that can catch local outliers missed by a linear PCA subspace. For multi-class fault classification, gradient-boosted trees—particularly XGBoost [11]—remain a strong tabular baseline on TEP because they handle feature interactions and nonlinear process behavior well.

Explainability in FDD has been pursued through local feature-attribution methods such as LIME [15] and SHAP (SHapley Additive exPlanations) [14]. SHAP attributes each model prediction to individual input variables in a game-theoretic framework. In process diagnosis, such attributions can identify which process variables contribute most to a fault classification, supporting root-cause analysis.

2.2. LLM-Augmented Fault Diagnosis

LLMs have been used for text generation and few-shot task adaptation [16, 17], but unsupported or inconsistent statements remain a known failure mode [18]. Work at the intersection of LLMs and process monitoring is still limited. [19] proposed FaultExplainer, which feeds the outputs of a trained fault classifier to an LLM to produce post-hoc explanations on TEP. FaultExplainer shows that such narratives can be useful, but the LLM receives pre-computed outputs passively; it does not call tools, adapt evidence gathering, or check each evidence field. EviFDD-Agent instead calls tools

sequentially, requires reported numeric values to match tool outputs, and counts unsupported claims at the report level through URR.

2.3. Tool-Augmented LLM Agents

Augmented language models combine LLMs with external tools, retrieval modules, or other non-parametric resources [21]. ReAct [20] interleaves generation with tool calls in a Thought–Action–Observation loop, while Toolformer [22] and ToolLLM [23] study broader API use. In process-related applications, LLM systems have been reported for process design [24] and materials discovery [25]. Less attention has been paid to whether an FDD report can be audited field by field against the computations that produced it.

2.4. Retrieval-Augmented Generation

RAG [26] adds retrieved documents to LLM generation, and recent surveys treat it as a standard way to supply external knowledge [27]. Here, RAG retrieves prior case descriptions from a structured TEP fault knowledge base and supplies process context for the narrative part of the report. It is only one evidence source; detection statistics, anomaly scores, classifier probabilities, and SHAP variables are supplied by separate tools. Recent C&CE work has combined LLMs, analytical toolboxes, and RAG for industrial control-loop oscillation analysis [28]. The present paper focuses instead on FDD reports whose structured fields can be checked against an evidence record.

3. Methodology

3.1. System Architecture

For a sensor observation $\mathbf{x} \in \mathbb{R}^{52}$, EviFDD-Agent enters a ReAct loop capped at 10 steps (Figure 1). At each step, the LLM emits a **Thought–Action–Action Input** triplet; the executor runs the named tool, appends the **Observation** to the message history, and returns the updated context to the LLM. The loop terminates only when a **Final Answer** JSON block is

produced *and* all five tool observations are present in the evidence record. If the LLM writes a report too early, the executor rejects it and forces the next missing tool call.

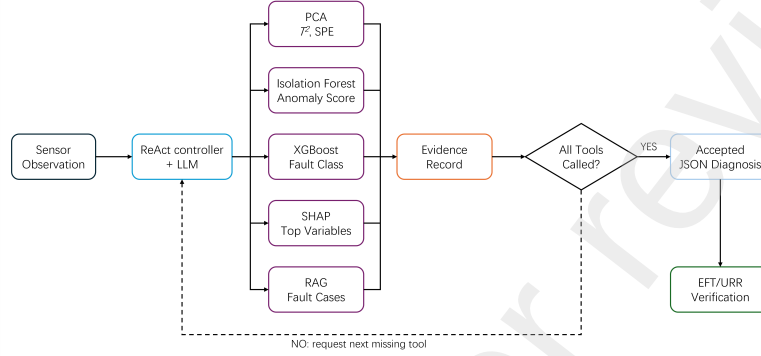


Figure 1: EviFDD-Agent architecture. The controller calls five diagnostic tools, stores their outputs in the Evidence Record, checks tool-call completeness before accepting a final JSON diagnosis, and then computes EFT/URR against the accepted report.

3.2. Diagnostic Tool Stack

Fault detection is based on a PCA monitor fitted to normal operating data. All 52 process variables are standardised to zero mean and unit variance using training-set statistics before PCA fitting and inference (i.e., $\tilde{\mathbf{x}} = (\mathbf{x} - \boldsymbol{\mu})/\boldsymbol{\sigma}$). A PCA model with $k = 10$ principal components is fitted on normal operation data ($n = 500$ samples, random seed 42). For a standardised observation $\tilde{\mathbf{x}}$, the T^2 statistic is:

$$T^2 = \tilde{\mathbf{x}}^\top \mathbf{P} \boldsymbol{\Lambda}^{-1} \mathbf{P}^\top \tilde{\mathbf{x}}, \quad (1)$$

where $\mathbf{P} \in \mathbb{R}^{52 \times k}$ contains the loadings and $\boldsymbol{\Lambda}$ is the diagonal eigenvalue matrix. The SPE statistic measures reconstruction error in the residual sub-space:

$$\text{SPE} = \|(\mathbf{I} - \mathbf{P} \mathbf{P}^\top) \tilde{\mathbf{x}}\|^2. \quad (2)$$

Control limits at significance level $\alpha = 0.01$ are derived via the F -distribution for T^2 and the Jackson–Mudholkar approximation for SPE [9]. A fault is

flagged if either statistic exceeds its limit. To complement the linear PCA monitor, we also train an Isolation Forest [10] with 200 trees and contamination rate 0.05 on normal data. The anomaly score $s(\mathbf{x}) \in [0, 1]$ is the normalized average path length, with $s > 0.5$ indicating an anomaly. The final detection decision uses the logical OR of the PCA and Isolation Forest flags.

Fault classification is handled by XGBoost [11]. The classifier is trained on labeled training data ($n = 500$ samples per fault class, 22 classes including normal, standardised with the same scaler as PCA). The model uses 300 trees, maximum depth 6, learning rate 0.1, subsampling ratio 0.8, column subsampling ratio 0.8, and random seed 42. The classifier returns the predicted fault class $\hat{y} \in \{0, 1, \dots, 21\}$ and class probability vector $\hat{\mathbf{p}}$, from which fault confidence $\hat{p}_{\hat{y}}$ is extracted.

Variable attribution uses SHAP TreeExplainer [14] on the trained XGBoost model. SHAP values $\phi_j(\mathbf{x})$ are computed for the predicted fault class, and the top-5 variables by $|\phi_j|$ are returned with their direction (increase/decrease). These variables form the causal attribution evidence used by the report generator.

Recommendation text is supported by a compact retrieval module. A structured knowledge base of 12 TEP fault cases is indexed in a ChromaDB vector store with a TF-IDF fallback. The module is used as a controlled evidence source, not as a complete mechanistic knowledge base. Given the predicted fault class from the classifier, the retriever returns the top- $k = 3$ case summaries. The cases cover IDV(1)–(7) and IDV(11)–(15), written by the authors from public TEP benchmark documentation [4, 29]. For IDV(8)–(10) and IDV(16)–(21), the retriever returns the nearest available case; the `rag_case` EFT field therefore verifies citation fidelity to the retrieved record, not the mechanistic correctness of that record for the true fault. Supplementary Table S3 lists these out-of-knowledge-base retrievals.

3.3. Evidence Citation Schema and Evidence Field Traceability

After all five tools execute, their outputs are stored in an evidence record $\mathcal{E}$:

$$\mathcal{E} = \{T^2, \text{SPE}, s_{\text{IF}}, \hat{y}, \hat{p}_{\hat{y}}, \{\phi_j\}_{j=1}^5, \text{RAG case}\}. \quad (3)$$

The seven indexed evidence fields are $f_1 = T^2$, $f_2 = \text{SPE}$, $f_3 = s_{\text{IF}}$, $f_4 = \hat{y}$, $f_5 = \hat{p}_{\hat{y}}$, $f_6 = \{\phi_j\}_{j=1}^5$, and $f_7 = \text{RAG case}$.

The LLM then produces a structured JSON report $\mathcal{R}$ with seven required evidence fields: $\{\hat{T}^2, \widehat{\text{SPE}}, \hat{s}, \hat{y}_{\text{LLM}}, \hat{p}_{\text{LLM}}, \hat{\phi}, \text{RAG}_{\text{LLM}}\}$.

For each required field f_i , a verification function v_i checks whether the reported value is grounded in the corresponding evidence value. Evidence Field Traceability is defined as:

$$\text{EFT}(\mathcal{R}, \mathcal{E}) = \frac{1}{7} \sum_{i=1}^7 v_i(\mathcal{R}, \mathcal{E}), \quad (4)$$

where $v_i \in \{0, 1\}$. The verification rules per field type are:

- *Numeric fields* (T^2 , SPE, anomaly score, fault confidence): $v_i = 1$ iff $|r_i - e_i|/|e_i| \leq \varepsilon = 0.05$ (5% relative tolerance). If $e_i = 0$, require $|r_i| < 10^{-6}$.
- *Categorical field* (fault class): $v_i = 1$ iff $r_i = e_i$ (exact integer match).
- *Set-valued field* (top variables): $v_i = 1$ iff at least one variable identifier in the report's SHAP list appears in the evidence SHAP list by exact string equality (e.g., $\text{XMEAS}(1)$, $\text{XMANIP}(3)$).
- *Text-grounded field* (RAG case): $v_i = 1$ iff the exact case identifier returned by the RAG tool appears in the report's `rag_case` field.

If evidence field $\mathcal{E}_i$ is absent (tool was not called), $v_i = 0$ and an unsupported-field flag is raised. EFT ranges from 0 (no grounding) to 1 (all fields verified). EFT is a structured evidence-compliance metric. It checks whether required report fields are traceable to tool observations; free-text quality is evaluated separately in Section 4.5.

At the report level, URR measures whether any required field in a diagnosis fails verification. Over a test set of N samples:

$$\text{URR} = \frac{1}{N} \sum_{n=1}^N \mathbf{1}[\exists i : v_i(\mathcal{R}^{(n)}, \mathcal{E}^{(n)}) = 0]. \quad (5)$$

URR is therefore the fraction of diagnoses containing at least one required structured field that is not traceable to the evidence record. We separate three error types: Type I (citation/transcription error), where $\mathcal{R}_i \neq \mathcal{E}_i$ even though $\mathcal{E}_i$ exists; Type II (tool prediction error), where $\mathcal{E}_i \neq y^*$ because the tool is wrong relative to ground truth; and Type III (unpopulated evidence), where the report gives a value for a required field that no tool populated. URR counts Type I and Type III errors. Type II errors are captured by E2E-Acc. The Tool-Free baseline’s $\text{URR} = 1.00$ is a Type III failure, while the Passive and Agent baselines mainly expose Type I failures. $\text{URR} = 0$ means every required structured report field is traceable to a tool observation, not that the tool prediction is correct or that the free-text recommendation is sufficient.

EFT and URR use different denominators. EFT averages over all $N \times 7$ field-sample pairs, while URR counts a sample as failed if any one field fails. A report with one failed field has $\text{EFT} = 6/7 \approx 0.857$ but $\text{URR} = 1$, so high mean EFT can coexist with a nonzero URR. For the Flash run, three URR-positive reports contain four failed field checks, reducing mean EFT by $4/(210 \times 7) \approx 0.27\%$, which matches $\text{EFT} = 0.997$.

3.4. Preliminary Report Actionability Score

EFT and URR are restricted to required structured fields. As a lightweight check on whether generated text contains action-oriented report elements, we use a preliminary rule-based Report Actionability Score (RAS):

$$\text{RAS}(\mathcal{R}) = \frac{1}{7} \sum_{j=1}^7 a_j(\mathcal{R}), \quad (6)$$

where $a_j \in \{0,1\}$ denotes seven heuristic checks: nonempty diagnosis, nonempty operator action, exact SHAP-variable citation, overlap between the recommended action and retrieved RAG case, operational specificity, mechanism context, and support for high-risk actions. RAS is not an expert human score and does not establish operator usefulness; it is a reproducible first-pass screen for whether expected report elements are present.

3.5. ReAct Agent with Evidence Enforcement

Algorithm 1 formalizes the diagnosis loop. The enforcement mechanism operates at the *tool-call completeness* level (lines 8–13): if the LLM generates a **Final Answer** before all five tools have been invoked, the report is *rejected* and the agent injects a correction message listing the missing tools, forcing continued tool execution. EFT and URR are *evaluation metrics* computed after the final report is accepted; they are not used as runtime rejection criteria. The improvement in SHAP field fidelity arises from the *sequential context structure* of ReAct: the LLM processes each tool output in its own turn immediately before producing the corresponding report section, which reduces paraphrasing of variable identifiers. In passive single-prompt mode, all tool outputs arrive simultaneously and the LLM paraphrases them into natural language by default.

Algorithm 1 EviFDD-Agent Diagnosis Loop

Require: Sensor observation $\mathbf{x}$, tools $\mathcal{T}$, LLM π , max steps S

Ensure: Diagnosis report $\mathcal{R}$ with EFT

```
1: Initialize evidence record  $\mathcal{E} \leftarrow \emptyset$ ; message history  $\mathcal{M} \leftarrow$   
   [system prompt, user( $\mathbf{x}$ )]  
2: for  $s = 1, \dots, S$  do  
3:   output  $\leftarrow \pi(\mathcal{M})$ ; append output to  $\mathcal{M}$   
4:   if Final Answer detected in output then  
5:     if  $\mathcal{E}$  is complete (all 5 tools called) then  
6:       break  
7:     else  
8:       missing  $\leftarrow \{t \in \mathcal{T} : \mathcal{E}_t = \emptyset\}$   
9:       Append rejection message to  $\mathcal{M}$ : "REJECTED. Call  
   missing[0] now."  
10:      continue  
11:    end if  
12:  end if  
13:  Parse tool name  $t$  and input  $a$  from output  
14:  obs  $\leftarrow t.execute(\mathbf{x}, a)$ ; update  $\mathcal{E}$  with obs  
15:  Append observation to  $\mathcal{M}$   
16: end for  
17:  $\mathcal{R} \leftarrow \text{parse\_json}(\text{output})$ ; compute EFT( $\mathcal{R}, \mathcal{E}$ )  
18: return  $\mathcal{R}$ 
```

The system prompt specifies a fixed tool order, prohibits final-answer generation before all tool observations are available, and instructs the model to copy structured evidence fields verbatim. The model is not told the numerical value of the step limit, but the executor enforces a maximum of 10 ReAct steps. EFT measures evidence-field compliance under this protocol rather than independent model reasoning. A representative prompt excerpt is provided in Supplementary Table S1, and the complete prompt is included in the released source code.

4. Experiments

4.1. Dataset: Tennessee Eastman Process

The Tennessee Eastman Process (TEP) [4] is the canonical benchmark for industrial FDD. It simulates a chemical plant with 52 process variables (41 measurements + 11 manipulated variables), 1 normal operating mode (IDV(0)), and 21 fault scenarios (IDV(1)–IDV(21)) representing step changes, random variations, slow drifts, and valve sticking. Each fault class has 500 training samples and 960 test samples.

Post-injection evaluation. Samples at $t \leq 160$ in each test fault trajectory correspond to the pre-injection period, when the process is still running normally. Including these samples in classification evaluation inflates accuracy artificially. We apply a *post-injection mask*: only samples at time steps $t > 160$ are used for classification and agent evaluation. This yields 800 post-injection test samples per fault class (16,800 samples total for classification, 17,760 including IDV(0)).

LLM experiment sampling. Due to API cost constraints, the constrained agent experiments sample a fixed number of observations per fault class from the post-injection pool using fixed random seeds for reproducibility. Table 1 summarizes the evaluation protocol.

Table 1: Evaluation protocol for the main experiments.

Experiment	Fault classes	Sampling	Backend	Paired comparison
Tool-free LLM sanity check	IDV(1)–IDV(21)	10/fault, seed 42 ($n = 210$)	DeepSeek-V4-Flash	No matched comparison
Template report generator	IDV(1)–IDV(21)	10/fault, seed 42 ($n = 210$)	No LLM	Passive baseline
Passive LLM baseline	IDV(1)–IDV(21)	10/fault, seed 42 ($n = 210$)	DeepSeek-V4-Flash	Template report generator
Report-content heuristic check	IDV(1)–IDV(21)	10/fault, seed 42 ($n = 210$)	Flash / no LLM	Template, exact-table, validator, ReAct
EviFDD-Agent Flash	IDV(1)–IDV(21)	10/fault, seed 42 ($n = 210$)	DeepSeek-V4-Flash	Deterministic pipeline on same subset
EviFDD-Agent Pro	IDV(1)–IDV(21)	10/fault, seed 42 ($n = 210$)	DeepSeek-V4-Pro	Deterministic pipeline on same subset
Ablation study	IDV(1)–IDV(21)	200/fault deterministic subset	No LLM	Full tool stack

4.2. Fault Detection Performance

Adding Isolation Forest mainly improves sensitivity rather than precision. On the post-injection test set, PCA+IF raises FDR from 73.9% to 80.5% and improves F1 to 0.885, but FAR also increases from 12.2% to 25.8% (Table 2). This is the expected trade-off of the OR fusion rule: a sample is flagged when either PCA or Isolation Forest triggers.

Table 2: Fault detection performance on TEP test set (post-injection, $\alpha = 0.01$).

Method	FDR $\uparrow$	FAR $\downarrow$	Precision $\uparrow$	F1 $\uparrow$
PCA	0.7388	0.1219	0.9907	0.8464
IsolationForest	0.6371	0.2000	0.9824	0.7729
PCA+IF (ours)	0.8051	0.2583	0.9820	0.8848

The gain is uneven across faults. IDV(1), IDV(2), IDV(4)–IDV(7), IDV(14), and IDV(17) are consistently detected ($\text{FDR} > 0.95$), whereas IDV(3) and IDV(9) remain difficult (Figure 2), consistent with earlier TEP studies [29].

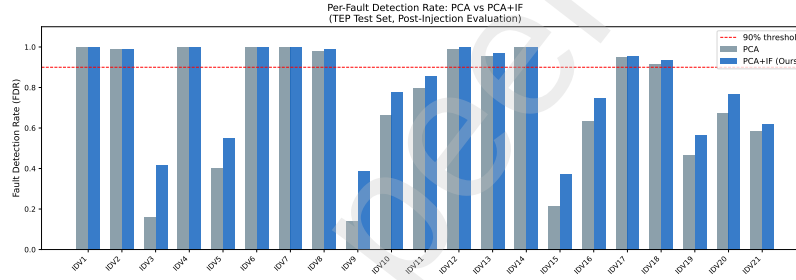


Figure 2: Per-fault detection rate on the TEP post-injection test set. PCA+IF improves sensitivity over PCA for several hard faults; dashed line: 90% FDR.

4.3. Fault Classification Accuracy

XGBoost achieves 66.6% overall accuracy ($\text{F1-macro} = 66.9\%$) on post-injection samples. IDV(1), IDV(2), IDV(5), IDV(6), IDV(7), and IDV(14) achieve $\text{F1} > 0.94$, reflecting their distinctive sensor signatures, whereas IDV(3), IDV(9), IDV(13), and IDV(15) have $\text{F1} < 0.25$, consistent with the known difficulty of these fault modes. Detailed per-fault precision, recall, and F1 scores are provided in Supplementary Table S2. The confusion matrix (Figure 3) reveals characteristic misclassification patterns: IDV(3) and IDV(15) are frequently confused with IDV(10) and IDV(9), reflecting their similar temperature-driven dynamics.

The classifier serves as the deterministic diagnostic backbone rather than a new state-of-the-art TEP classifier. Recent TEP classifiers that combine feature engineering, optimization, or deep sequence models report substantially higher headline performance under their own protocols, including XGB-AVSSA-KELM [12] and published deep sequence-model comparisons on TEP [13]. The present evaluation focuses on report traceability and preliminary report-content checks; paired accuracy checks in Section 4.7 verify that report generation preserves the backbone decision.

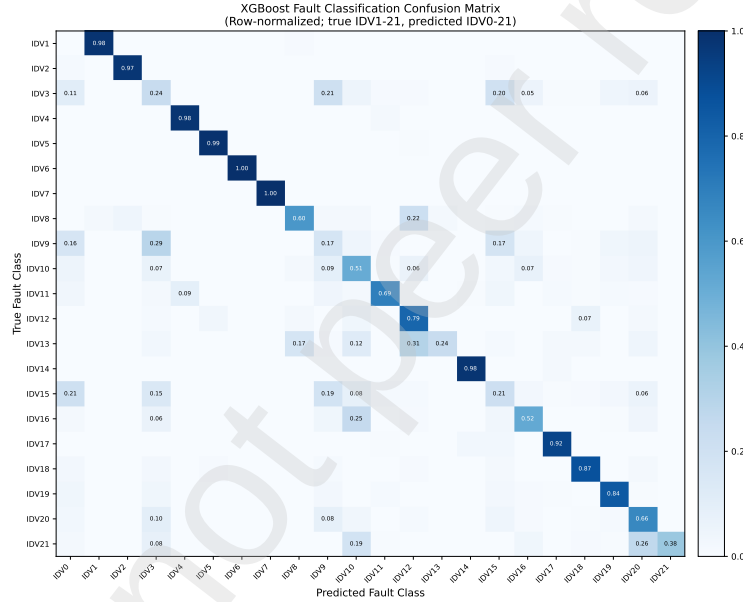


Figure 3: Row-normalized XGBoost confusion matrix on the TEP post-injection test set. Rows are true fault classes and columns are predicted classes; annotated cells exceed 0.05.

4.4. Evidence Traceability of Generated Diagnostic Reports

The evidence-fidelity experiments compare raw-sensor prompting with progressively more constrained access to diagnostic tool outputs. Table 3 summarizes the seven configurations.

Tool-free sanity check. Raw sensor prompting does not produce evidence-consistent reports in this setup. Without tool outputs, the LLM still writes

Table 3: Evidence Field Traceability (EFT) and Untraceable Report Rate (URR) for seven report-generation configurations on the TEP benchmark. URR 95% CI uses the Wilson score interval. Superscripts identify the evaluation protocol described in the text.

Configuration	n	EFT	EFT Std	URR	URR 95% CI
Tool-Free LLM Sanity Check ^a	210	N/A	N/A	1.000	[0.982, 1.000]
Template Report Generator ^b	210	1.000	0.000	0.000	[0.000, 0.017]
Passive LLM Baseline (Flash) ^c	210	0.890	0.060	0.771	[0.710, 0.823]
Exact-Table LLM (Flash) ^d	210	1.000	0.000	0.000	[0.000, 0.018]
Validator-Regeneration LLM (Flash) ^d	210	1.000	0.000	0.000	[0.000, 0.018]
EviFDD-Agent + DeepSeek-V4-Flash ^e	210	0.997	0.024	0.014	[0.005, 0.041]
EviFDD-Agent + DeepSeek-V4-Pro ^e	210	0.958	0.102	0.195	[0.147, 0.254]

^a Tool-free free-text diagnosis on 10 representative samples per TEP fault mode; because no tool observations are available to the LLM, all required evidence fields are treated as untraceable Type III fields. ^b Deterministic report generated directly from structured tool outputs. ^c Single-pass LLM report generated from the same evidence bundle without enforced tool use. ^d Competitive schema-control baseline using an exact evidence table; validator-regeneration adds a post-hoc checker. ^e ReAct agent with mandatory tool observations before final report generation.

structured evidence fields, but the reported T^2 and SPE values are untraceable in all 210 cases. The overall URR is 1.00. Because no tool evidence exists, this condition is not used as a matched performance baseline; it only shows how URR treats unsupported Type III fields.

Template report. The deterministic template serves as an upper-bound reference for fixed-field schema compliance. It fills the JSON report directly from the five tool outputs, giving $EFT = 1.000$ and $URR = 0.0\%$ ($n = 210$).

Passive baseline. Supplying all tool outputs in a single prompt is not enough to preserve all evidence identifiers. The Flash passive baseline has $URR = 77.1\%$ [71.0 %, 82.3%]. This failure is highly field-specific rather than a general numeric hallucination: T^2 , SPE, anomaly score, fault class, confidence, and RAG case all have zero URR, while the errors are concentrated in `top_variables` (Table 4). The model often replaces exact XMEAS/XMANIP identifiers with phrases such as “reactor temperature” or “separator pressure”. In an industrial report, the tag identifier links the text to the plant historian, tag list, SHAP record, and audit trail.

Exact-table and validator baselines. When the same evidence is presented as an exact JSON table with a strict copy prompt, Flash reaches $EFT = 1.000$ and $URR = 0.0\%$. Adding post-hoc validator-regeneration gives the same result, with a mean of one attempt per report. These baselines show that evidence serialization largely determines schema compliance. Their mean token usage is about 1.16k–1.18k tokens per diagnosis and mean wall-clock latency is about 6.3–6.4s in this run.

Flash agent. With ReAct enforcement, Flash approaches the template’s schema compliance while retaining generated narrative fields: $EFT = 0.997$ and $URR = 1.4\%$ at $n = 210$. The useful contrast is evidence fidelity: sequential tool use reduces the passive baseline’s report-level URR from 77.1% to 1.4%. The three remaining Flash failures involve either formatted SHAP-variable strings that no longer exactly match the evidence identifiers or a classifier-field mismatch in one report.

Pro agent. The same protocol is less reliable with DeepSeek-V4-Pro: $EFT = 0.958$ and $URR = 19.5\%$. The per-field breakdown in Table 4 localizes the difference: Pro’s largest failure channel is `top_variables` (15.7%), followed by RAG case citation (4.8%) and smaller numeric-field failures for T^2 and SPE (2.9% each). The typical pattern is not lack of evidence access, because the tools were called, but loss of exact copy fidelity: Pro often inserts SHAP values or parentheses into variable names, or omits a required cited case after a longer reasoning trajectory.

The dominant failure mode is not lack of evidence access, but the serialization of evidence presented to the LLM. Table 5 compares the resulting design space. The passive LLM loses machine-checkable SHAP identifiers despite receiving the same tool evidence, while the exact-table and validator baselines remove identifier-level URR through explicit serialization. ReAct adds a process-level audit mechanism by exposing the diagnostic tool trajectory; exact-table prompting remains the lower-cost route when the evidence bundle has already been assembled.

Table 4: Per-field Untraceable Report Rate (URR). Each cell shows the fraction of samples in which the corresponding evidence field fails EFT verification ($n = 210$ for all rows).

Configuration	T^2	SPE	Anom. Score	Fault Class	Fault Conf.	Top Vars	RAG Case
Tool-Free LLM Sanity Check	1.000	1.000	1.000	1.000	1.000	1.000	1.000
Template ^a	0.000	0.000	0.000	0.000	0.000	0.000	0.000
Passive LLM ^b	0.000	0.000	0.000	0.000	0.000	0.771	0.000
Exact-Table LLM ^c	0.000	0.000	0.000	0.000	0.000	0.000	0.000
Validator-Regeneration LLM ^c	0.000	0.000	0.000	0.000	0.000	0.000	0.000
EviFDD-Agent Flash ^d	0.000	0.000	0.000	0.005	0.005	0.010	0.000
EviFDD-Agent Pro ^d	0.029	0.029	0.005	0.014	0.014	0.157	0.048

^a $n = 210$, seed=42. ^b $n = 210$, seed=42, DeepSeek-V4-Flash. ^c $n = 210$, seed=42, DeepSeek-V4-Flash, exact evidence table prompt. ^d Dedicated 210-sample per-field rerun with mandatory ReAct tool observations before final report generation.

Table 5: Evidence-interface trade-off across deterministic, passive, schema-controlled, and ReAct configurations.

Method	Evidence interface	Tool trace	EFT	URR	Main interpretation
Template	fixed fields	precomputed tools	1.000	0.000	Upper-bound schema compliance with fixed text.
Passive LLM	free-form block	precomputed tools	0.890	0.771	Loose prompting preserves numbers but breaks tag identifiers.
Exact-table LLM	JSON evidence table	precomputed tools	1.000	0.000	Explicit serialization solves structured-field compliance.
Validator-regeneration	JSON table + checker	precomputed tools	1.000	0.000	Adds a checker around generated JSON.
ReAct Flash	stepwise observations	visible tool trajectory	0.997	0.014	Adds auditable tool use with moderate orchestration cost.
ReAct Pro	stepwise observations	visible tool trajectory	0.958	0.195	Slower backend is less stable for exact evidence copying.

Table 6: Backend runtime and evidence-fidelity comparison for the dedicated 210-sample ReAct per-field rerun. Wall time includes sequential LLM calls, local tool execution, and provider latency.

Backend	EFT	URR	Top Vars URR	Total wall time	Mean s/sample	Mean tokens/sample
DeepSeek-V4-Flash	0.997	0.014	0.010	0.70 h	12.0	10.3k
DeepSeek-V4-Pro	0.958	0.195	0.157	8.26 h	141.5	21.8k
Pro / Flash ratio	N/A	13.7×	15.7×	11.8×	11.8×	2.1×

4.5. Heuristic Report-Content Checks Beyond Schema Compliance

EFT/URR evaluate structured evidence traceability. The narrative and recommendation fields add a separate surface: whether the report contains expected action-oriented elements after the evidence fields have been copied. The preliminary rule-based RAS, defined in Section 3.4, is computed on 210 samples (10 per fault class, seed 42). It is a reproducible content-completeness check, not a substitute for expert operator scoring.

Table 7: Preliminary rule-based Report Actionability Score (RAS) as a content-completeness screen for generated diagnostic narratives and recommendations. RAS is the mean pass rate over narrative non-emptiness, action presence, exact variable citation, RAG-action overlap, operational specificity, mechanism context, and high-risk-action support; it is not an expert operator score.

Method	n	RAS	Action	Variable	RAG-action	Specific	Mechanism	EFT
Template	210	0.571	0.000	1.000	0.000	0.000	1.000	1.000
Exact-table LLM	210	0.918	1.000	0.710	1.000	1.000	0.809	1.000
Validator-regeneration	210	0.908	1.000	0.714	1.000	1.000	0.752	1.000
ReAct Agent	210	0.935	1.000	0.876	0.924	0.995	0.824	0.999

The deterministic template obtains perfect EFT but a lower RAS (0.571) because it has no operator recommendation field. Exact-table and validator-regeneration prompting add the expected recommendation elements (RAS = 0.918 and 0.908, respectively) while preserving structured fields. In this rule-based check, the ReAct agent has the highest pass rate (RAS = 0.935), mainly because it preserves exact variable identifiers more often while still producing action-oriented recommendation text. The claim is limited to the predefined elements checked by RAS; human evaluation would be needed to assess operator value.

4.6. RAG Retrieval for Out-of-Knowledge-Base Faults

Supplementary Tables S3 and S6 list the retrieved cases and summarize retrieval coverage. The 12-case knowledge base covers 57.1% of the TEP fault classes. For covered faults, TF-IDF retrieval returns the exact case at rank 1 for all 12 classes and within the top three for all 12 classes. For uncovered classes, the RAG field is interpreted as citation to the retrieved case. Retrieval is plausible for IDV(21), where valve-related cases are returned, and

partly useful for IDV(9), where the exact D-feed temperature step-change case appears in the top three. IDV(16)–IDV(20), whose labels are “unknown fault”, collapse to the same nearest-neighbor pattern IDV(6), IDV(12), and IDV(13), which marks the current boundary of the compact knowledge base.

4.7. Operational Logging

The original LLM runs saved aggregate EFT/URR/accuracy values. The released code now extends this logging to ReAct steps, tool calls, rejection messages, wall-clock time, available API token usage, per-field verification, and fault identifier for each diagnosis. Each deterministic diagnosis uses five local tool calls; the remaining overhead comes from the sequential ReAct conversation and depends on provider latency and token accounting. PCA control chart (Figure 4). Figure 4 illustrates T^2 and SPE trajectories for an IDV(4) test sample. Both statistics are well within control limits before fault injection at $t = 160$; the SPE exceeds its limit immediately at $t = 160$, while the T^2 chart shows elevated exceedances from $t \approx 200$ onward. This visual shows how the PCA tool contributes detection evidence within the agent pipeline.

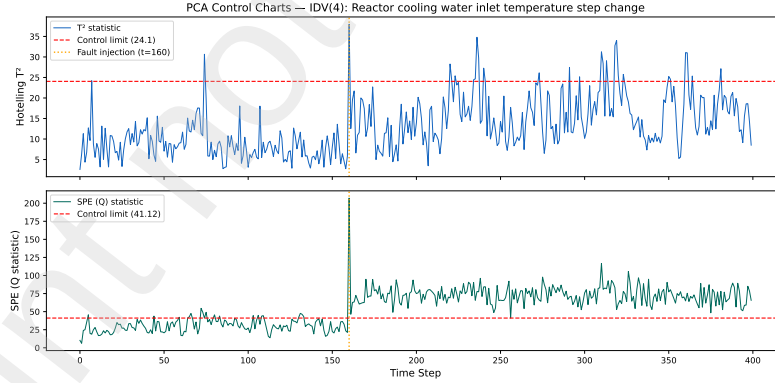


Figure 4: PCA monitoring evidence for an IDV(4) test trajectory. SPE crosses its control limit at fault injection; T^2 exceedances become pronounced after $t \approx 200$.

4.8. Paired Accuracy Comparison

Accuracy is checked against the deterministic pipeline on the *identical* sample subsets to confirm that evidence enforcement preserves the classifier decision (Table 8).

Table 8: Paired accuracy comparison: Deterministic Tool Pipeline vs. EviFDD-Agent evaluated on identical sample subsets ($n = 210$, 10 samples per fault class $\times$ 21 faults). Acc 95% CI uses the Wilson score interval.

Subset	n	Det. Acc	Det. 95% CI	Agent Acc	Agent 95% CI	Δ Acc
Flash	210	0.7048	[0.640, 0.762]	0.7095	[0.645, 0.767]	+0.48 pp
Pro	210	0.7048	[0.640, 0.762]	0.7000	[0.635, 0.758]	−0.48 pp

The agent is intended to preserve the classifier’s diagnostic decision while improving report auditability. On the shared 210-sample subset, Flash gives 70.95% accuracy and Pro gives 70.00%, compared with 70.48% for the deterministic pipeline on the same samples (Table 8). These small differences are consistent with the intended accuracy-preservation behavior: the LLM layer mainly affects report serialization and traceability, not the underlying classifier decision.

4.9. Evaluation Subset Representativeness

We checked whether the 210-sample evaluation subset (10/fault, seed=42) over-represents easy-to-classify faults. Supplementary Table S4 gives the per-class comparison.

The XGBoost full-test accuracy of 66.6% is computed on all 17,760 post-injection samples including IDV(0) (normal class, $n = 960$, recall = 0.181), which the agent evaluation subset excludes (the agent is designed for fault diagnosis, not normal/fault discrimination). Restricting to fault classes IDV(1)–IDV(21) only, the full-test accuracy is 69.3% ($n = 16,800$). The 210-sample subset achieves 70.5%—a gap of +1.15 pp relative to the fault-only full test set. The expected mean-accuracy standard deviation for a 10-sample-per-class subset is $\sigma \approx 0.022$ (derived from per-class std = $\sqrt{p(1-p)/10}$ averaged over the 21 fault classes); the observed +1.15 pp gap corresponds to $+0.53\sigma$, well within the range consistent with random sampling.

Per-class inspection (Supplementary Table S4) shows no enrichment of easy fault classes. The hard faults IDV(3), IDV(13), and IDV(15) each have recall = 0.200 in the subset, close to their full-set recalls of 0.240, 0.240, and 0.214. A complementary difficulty-stratified analysis (Supplementary Table S5) shows the expected separation: the evidence-fidelity subset reaches 96.4% accuracy on easy faults, 55.0% on medium faults, and 22.5% on hard faults. Thus the report-fidelity experiments include the classifier’s difficult regimes, not only clean cases. The largest per-class deviations (IDV9: +0.13; IDV16: −0.22; IDV21: +0.22) are consistent with $n = 10$ Bernoulli fluctuations.

4.10. Tool-Ablation Analysis

Removing tools clarifies which part of the pipeline supplies each kind of evidence (Table 9). This ablation is a schema-level accounting experiment: the deterministic report is populated only from available tool outputs, and EFT is the resulting fraction of required fields present. It is directly comparable with the template row in Table 3; with all tools present, both give EFT = 1.000.

Table 9: Schema-level tool ablation.

Configuration	Accuracy	EFT
Full (all tools)	0.7048	1.0000
Without PCA	0.7048	0.7143
Without Isolation Forest	0.7048	0.8571
Without Classifier	0.0000	0.5714
Without SHAP	0.7048	0.8571
Without RAG	0.7048	0.8571
Classifier only	0.7048	0.2857

The ablation is dominated by the XGBoost classifier: removing it reduces accuracy to 0% and EFT to 0.571. The retained four fields are T^2 , SPE, Isolation Forest anomaly score, and a nearest-neighbor RAG case; classifier-conditioned fault class, confidence, and SHAP fields are absent. Removing

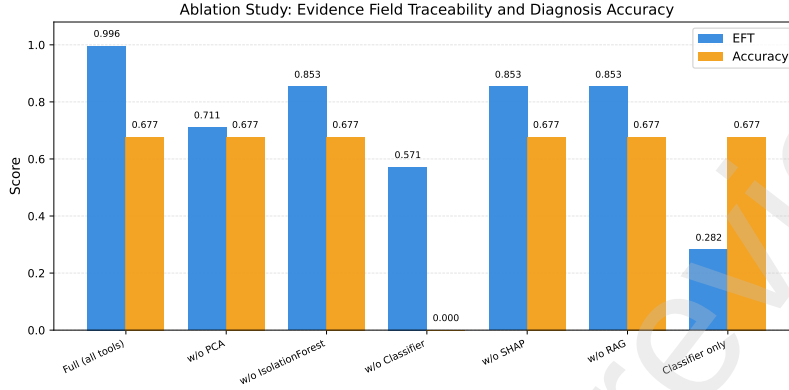


Figure 5: Tool-ablation results for EFT and diagnosis accuracy. Classification accuracy depends on XGBoost, while PCA, SHAP, and RAG add evidence fields.

PCA lowers EFT to 0.714 by dropping the T^2 /SPE fields but leaves classification accuracy unchanged. Removing Isolation Forest, SHAP, or RAG each lowers EFT to 0.857 by removing one of seven required evidence fields. The “Classifier only” configuration (EFT = 0.286) shows what is missing from a stand-alone classifier: five of the seven schema fields are absent.

5. Discussion

5.1. Evidence Grounding in Diagnostic Reporting

In operator-facing FDD reports, the risk is not only an incorrect class label. A report can also be dangerous when its explanation looks plausible but its numerical evidence cannot be traced back to the current process data. EFT and URR target this gap. The metrics focus on structured evidence fields because those fields can be checked automatically against tool outputs and plant tags.

5.2. Why an LLM Agent Rather Than a Template?

The template baseline is the natural reference point for fixed-field reporting: if the desired output is a JSON record with known fields, deterministic filling is straightforward. The exact-table and validator-regeneration baselines sharpen this point further. Once the evidence is packaged as a clean

JSON table, a single strict schema prompt reaches perfect structured-field compliance in this experiment. EviFDD-Agent addresses the next layer of the workflow, where evidence is gathered through explicit tool calls, intermediate observations are visible, and computed evidence, attribution, retrieved process knowledge, and natural-language explanation are assembled into an operator-facing report.

The practical lesson is that LLMs should not sit between critical evidence and locked structured fields. In a deployment-oriented design, deterministic tools or schema-controlled serializers should populate the auditable evidence layer; the LLM should operate on top of that layer to generate narrative explanation, recommendations, and interactive tool-use traces.

RAS has a narrower role than EFT/URR. EFT/URR establish evidence-traceability behavior; Table 7 checks whether the generated text contains predefined action-oriented elements. LLM-based reports add recommendation text beyond the deterministic template, and the ReAct agent preserves exact variable citations most consistently in the 210-sample check. Expert operator scoring remains necessary for stronger claims about mechanism explanations, recommendation quality, uncertainty statements, and behavior under conflicting tool outputs.

5.3. Backend Differences in URR and Runtime

The Flash and Pro results differ mainly in transcription behavior, not in access to evidence. Flash tends to follow the evidence-copying protocol, whereas Pro more often changes the surface form of SHAP identifiers, omits the retrieved case citation, or rounds scalar values. The dedicated 210-sample per-field rerun makes this visible: Pro has `top_variables` URR = 0.157, while Flash has 0.010.

The backend difference is also large in cost terms. Flash completes the 210-sample ReAct run in 0.70 h (12.0s/sample on average), while Pro takes 8.26 h (141.5s/sample), an 11.8× wall-time increase (Table 6). Mean token usage is also higher for Pro (21.8k vs. 10.3k tokens/sample). Under the same evidence-copying protocol, Pro is both less traceable and substantially more expensive. Evidence-fidelity controls must be validated with the deployed

model, especially when exact plant-tag identifiers are required.

5.4. Accuracy Preservation and the Role of the LLM Layer

The agent is designed to improve auditability and explanation while preserving the diagnostic decisions of the underlying classifier. The XGBoost classifier sets the fault-identification behavior, and the LLM layer packages that decision with evidence, attribution, and retrieved process context. The paired comparisons show that the agent leaves the classifier’s decisions essentially unchanged, while EFT/URR evaluate traceability and RAS provides only a preliminary rule-based screen of action-oriented report elements.

5.5. Limitations

The current study is scoped to single-snapshot TEP diagnosis. The LLM experiments use 210 samples per backend, which supports aggregate comparisons while leaving per-class URR estimates noisy for hard faults such as IDV(3), IDV(9), IDV(13), and IDV(15). With only 10 samples per fault class, the per-class Wilson intervals are too wide to support strong fault-specific claims; the Flash URR of 1.4% is a low observed failure rate, not proof of optimality over nearby prompt or backend variants. Plant deployment would add temporal context, alarm persistence logic, and validation on processes beyond TEP.

The diagnostic backbone also limits engineering applicability. XGBoost is used as a fixed classifier to study report traceability, not as a production-grade TEP classifier. Its full post-injection accuracy is 66.6% overall and 69.3% on fault classes only, with weak F1 scores for IDV(3), IDV(9), IDV(13), and IDV(15). When the backbone predicts the wrong fault, the LLM can still produce a traceable report, but the traced decision is wrong. Practical deployment would require a stronger diagnostic model or tighter coupling between classification confidence and report generation.

The knowledge base is intentionally compact: it covers 12 TEP cases, so recommendations for uncovered or unknown faults rely on nearest-neighbor retrieval. For those classes, the RAG-related EFT field measures citation fidelity to the retriever output. Full deployment would require broader case

coverage and domain review of the recommendation library. The RAS experiment is a rule-based first pass over 210 generated reports; it should not be interpreted as evidence of operator value without expert human evaluation.

The competitive baselines clarify the architectural choice. Exact-table prompting and validator-regeneration achieved $URR = 0$ once all evidence had already been assembled into a clean schema. These approaches are attractive for fixed reports, whereas ReAct exposes the tool-use trace and supports a more interactive evidence-gathering workflow. The Pro result also shows that a larger or slower backend is not automatically preferable for evidence-copying tasks. Future work should compare these alternatives under matched prompts, latency, token budgets, and report-quality criteria.

5.6. Comparison with FaultExplainer

FaultExplainer [19] generates natural-language explanations by feeding a pre-trained classifier's output to an LLM in a single forward pass. EviFDD-Agent differs in three respects: (1) it invokes tools sequentially rather than receiving passive inputs; (2) it verifies every structured evidence field against tool evidence; and (3) it combines detection, classification, attribution, and retrieval, whereas FaultExplainer focuses on post-hoc explanation of one classifier.

6. Conclusions

This study examined evidence-traceable LLM reporting for industrial process fault detection and diagnosis. The workflow uses an Evidence Citation Schema: tools must be called before report generation, early final answers are rejected, and the accepted report is checked against the evidence record with EFT and URR.

Evidence-traceable reporting requires an explicit schema interface. A passive LLM can preserve the fault class while still losing machine-checkable SHAP identifiers. Exact-table prompting, validator-regeneration, and ReAct all largely solve this structured-field problem, but with different engineering trade-offs. The deterministic template remains the simplest option for fixed-field copying; the agent is useful when the report must combine structured

evidence with attribution, retrieved process context, narrative explanation, and an inspectable tool-use trace. The Flash-Pro comparison further shows that model selection must consider evidence fidelity and runtime together. The LLM is best treated as a constrained explanation and recommendation layer above deterministic evidence serialization, not as a transcriber of critical diagnostic evidence.

Future work should extend the report-content evaluation beyond rule-based RAS: streaming data, multi-fault cases, broader process benchmarks, complete knowledge-base coverage, expert scoring of recommendations, and direct comparison between ReAct, validator-regeneration, and provider-level structured output binding.

Funding: This work was supported by the Science and Technology Research and Development Plan Joint Fund (Industry) of Henan Province (No. 245101610004), the High-end Foreign Experts Recruitment Program of Henan Province (No. HNGD2023001), the High-end Foreign Experts Recruitment Program of China (No. G2023026020L), and the Key Research and Development Program of Henan Province (No. 241111220400).

Data Availability Statement: The source code, experiment scripts, and result summaries are available in a public [GitHub repository](#). Reproducible trained artifacts and cached public benchmark data are excluded from the repository and can be regenerated from the provided scripts.

Conflicts of Interest: The authors declare no conflict of interest.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process: During the preparation of this work, the authors used OpenAI ChatGPT and Codex for language editing, reference-checking assistance, LaTeX/code organization, and consistency checks. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

- [1] Venkatasubramanian, V.; Rengaswamy, R.; Yin, K.; Kavuri, S.N. A review of process fault detection and diagnosis: Part I: Quantitative

- model-based methods. *Comput. Chem. Eng.* **2003**, *27*, 293–311.
- [2] Venkatasubramanian, V.; Rengaswamy, R.; Kavuri, S.N. A review of process fault detection and diagnosis: Part II: Qualitative models and search strategies. *Comput. Chem. Eng.* **2003**, *27*, 313–326.
 - [3] Venkatasubramanian, V.; Rengaswamy, R.; Kavuri, S.N.; Yin, K. A review of process fault detection and diagnosis: Part III: Process history based methods. *Comput. Chem. Eng.* **2003**, *27*, 327–346.
 - [4] Downs, J.J.; Vogel, E.F. A plant-wide industrial process control problem. *Comput. Chem. Eng.* **1993**, *17*, 245–255.
 - [5] Qin, S.J. Survey on data-driven industrial process monitoring and diagnosis. *Annu. Rev. Control* **2012**, *36*, 220–234.
 - [6] Chiang, L.H.; Russell, E.L.; Braatz, R.D. *Fault Detection and Diagnosis in Industrial Systems*; Springer: London, UK, 2001.
 - [7] MacGregor, J.F.; Kourti, T. Statistical process control of multivariate processes. *Control Eng. Pract.* **1995**, *3*, 403–414.
 - [8] Ge, Z.; Song, Z.; Gao, F. Review of recent research on data-based process monitoring. *Ind. Eng. Chem. Res.* **2013**, *52*, 3543–3562.
 - [9] Jackson, J.E.; Mudholkar, G.S. Control procedures for residuals associated with principal component analysis. *Technometrics* **1979**, *21*, 341–349.
 - [10] Liu, F.T.; Ting, K.M.; Zhou, Z.H. Isolation forest. In *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining*; 2008; pp. 413–422.
 - [11] Chen, T.; Guestrin, C. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*; 2016; pp. 785–794.

- [12] Hu, M.; Hu, X.; Deng, Z.; Tu, B. Fault diagnosis of Tennessee Eastman process with XGB-AVSSA-KELM algorithm. *Energies* **2022**, *15*, 3198.
- [13] Kai, T.C.Y.; Saptoro, A.; Putra, Z.A.; Lim, K.H.; Yeo, W.S.; Sunarso, J. Supervised deep learning algorithms for process fault detection and diagnosis under different temporal subsequence length of process data. *Appl. Intell.* **2025**, *55*, 883.
- [14] Lundberg, S.M.; Lee, S.I. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*; 2017; Volume 30.
- [15] Ribeiro, M.T.; Singh, S.; Guestrin, C. “Why should I trust you?”: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*; 2016; pp. 1135–1144.
- [16] Brown, T.B.; Mann, B.; Ryder, N.; et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*; 2020; Volume 33, pp. 1877–1901.
- [17] Bubeck, S.; Chandrasekaran, V.; Eldan, R.; et al. Sparks of artificial general intelligence: Early experiments with GPT-4. *arXiv* **2023**, arXiv:2303.12712.
- [18] Ji, Z.; Lee, N.; Frieske, R.; et al. Survey of hallucination in natural language generation. *ACM Comput. Surv.* **2023**, *55*(12), Article 248, 1–38.
- [19] Khan, A.; Nahar, R.; Chen, H.; Constante Flores, G.E.; Li, C. FaultExplainer: Leveraging large language models for interpretable fault detection and diagnosis. *Comput. Chem. Eng.* **2025**, *199*, 109152.
- [20] Yao, S.; Zhao, J.; Yu, D.; et al. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*; 2023.

- [21] Mialon, G.; Dessì, R.; Lomeli, M.; et al. Augmented language models: A survey. *Transactions on Machine Learning Research* **2023**.
- [22] Schick, T.; Dwivedi-Yu, J.; Dessì, R.; et al. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*; 2023; Volume 36.
- [23] Qin, Y.; Liang, S.; Ye, Y.; et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*; 2024.
- [24] Boiko, D.A.; MacKnight, R.; Kline, B.; Gomes, G. Autonomous chemical research with large language models. *Nature* **2023**, *624*, 570–578.
- [25] Bran, A.M.; Cox, S.; Schilter, O.; et al. Augmenting large language models with chemistry tools. *Nat. Mach. Intell.* **2024**, *6*, 525–535.
- [26] Lewis, P.; Perez, E.; Piktus, A.; et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*; 2020; Volume 33, pp. 9459–9474.
- [27] Gao, Y.; Xiong, Y.; Gao, X.; et al. Retrieval-augmented generation for large language models: A survey. *arXiv* **2023**, arXiv:2312.10997.
- [28] Singh, A.; Modir Roustaa, M.; Huang, B. Oscillation analysis using Retrieval-Augmented Generation. *Comput. Chem. Eng.* **2026**, *206*, 109489.
- [29] Bathelt, A.; Ricker, N.L.; Jelali, M. Revision of the Tennessee Eastman process model. *IFAC-PapersOnLine* **2015**, *48*, 309–314.